

Spring :->

What is Spring framework ->

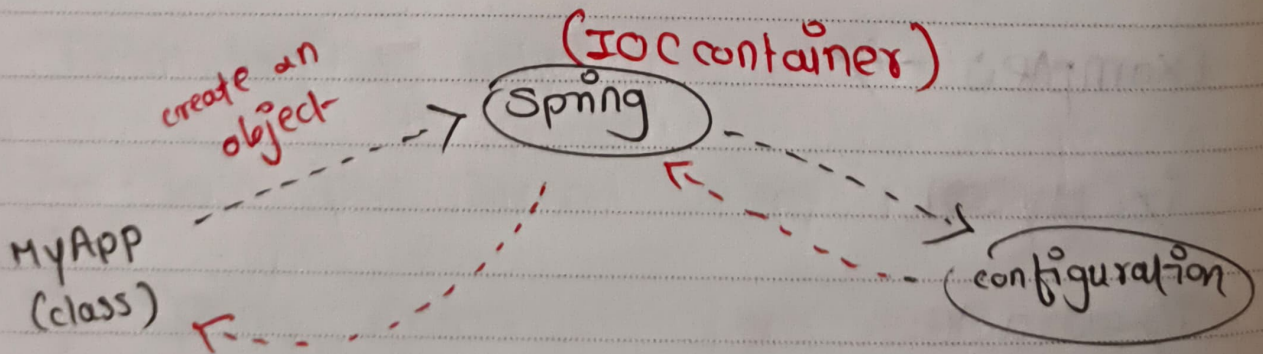
The Spring framework is a powerful, lightweight, and open source framework for building Java application.

It is the framework that helps the developers to work on their application rather than worrying about non-functional code.

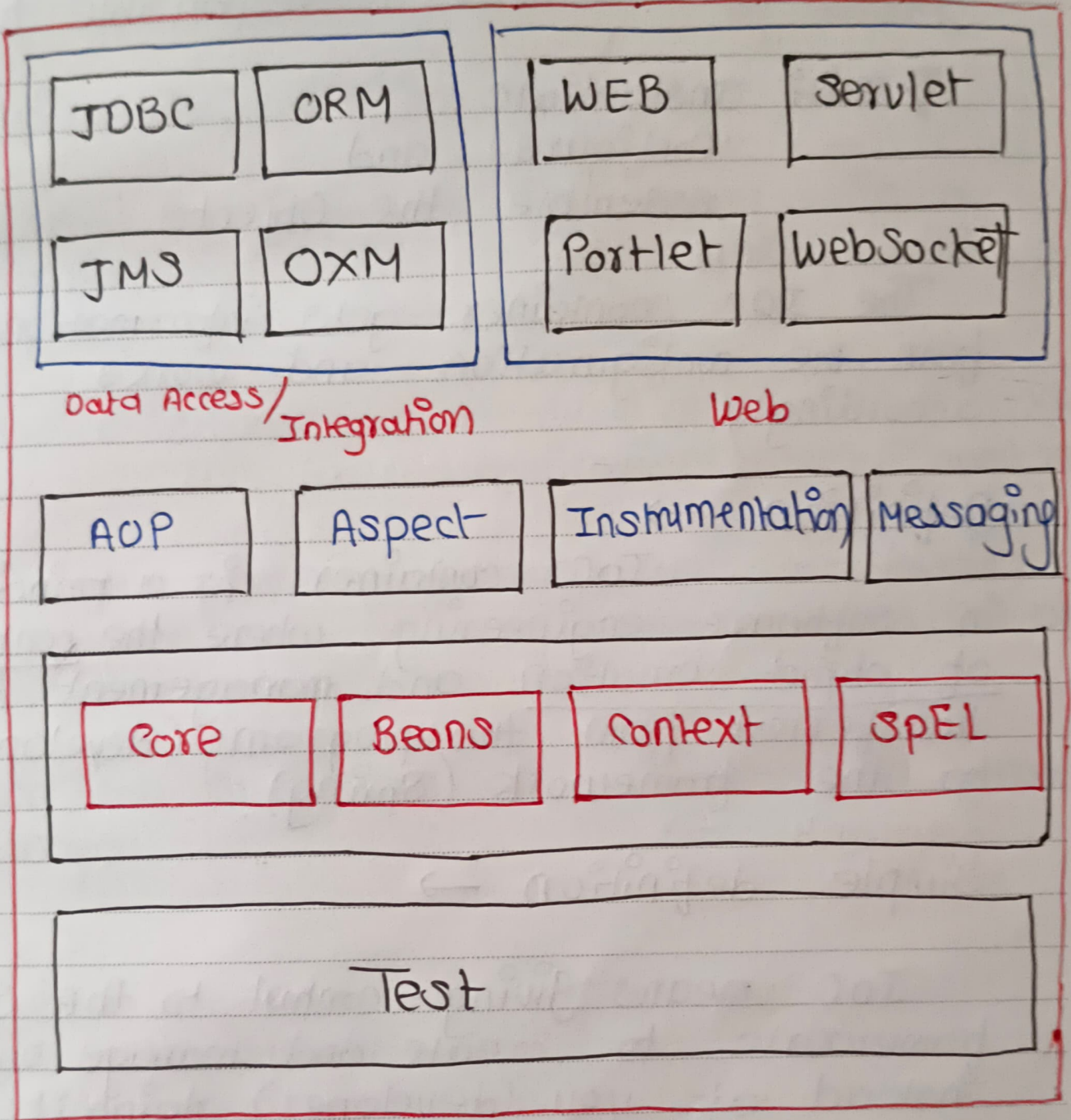
So, the developer can focus on business logic rather than non-functional requirement.

It minimized boilerplate java code.

Spring is a dependency injection framework to make java application loosely coupled.



Spring modules →



IOC container $\Rightarrow$ (Inversion of control)

The IOC container is responsible to

• **Initi** Instantiate
Configure and
Assemble the Objects

The IOC container gets information from the configuration and works accordingly.

Definition $\rightarrow$

IOC container is a principle in software engineering where the control of object creation and management is transferred from the program (developer) to the framework (Spring).

Simple definition $\rightarrow$

• IOC means giving control to the Spring framework to create and manage objects instead of you (developer) doing it manually.

* without IOC →

Car c = new Car() // we create it (object) by ourself

* With IOC →

Need to define a car bean in a Configuration file or annotation and Spring will create it for you.

ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");

Car car = context.getBean("car", Car.class);
// Spring created an object

* We can configure IOC container in following ways

i) XML

ii) Annotations.

iii) Java Configuration Class.

IOC Container Achieved in Spring By →

There are two main Containers.

- ① BeanFactory
- ② ApplicationContext.

① BeanFactory →

i) Lightweight Container.

ii) Basic Implementation of IOC.

iii) Loads beans lazily (only when requested)

iv) Suitable for simple applications.

Example →

```
BeanFactory factory = new XmlBeanFactory  
(new FileSystemResource  
("beans.xml"));
```

```
Car car = (Car) factory.getBean("car");
```

(2) ApplicationContext →

- i) Advanced container.
- ii) Extends BeanFactory.
- iii) Loads beans eagerly (at startup)
- iv) Provides extra features like :
 - Internationalization.
 - Event propagation.
 - AOP support.
 - Annotation scanning.

Example →

```
ApplicationContext context = new ClassPathXml
    ApplicationContext("bean.xml");

Car car = (Car) context.getBean("car");
```


Now lets understand with code →

- ① Create vehicle Interface with method getMileage.
- ② Create class Bus implements vehicle
- ③ Create class Car implements vehicle
- ④ In Main class create

```
Vehicle vehicle = new Bus();
```

} so here we need to hardcore ^{or} car();
the object name i.e. Bus() or Car()

But with the help of Spring IOC Containers we can create object with Configuration

}

Now lets understand how we can create object using IOC

- ① we need to create an config.xml file
- ② In that config.xml create an bean (object of class)

config.xml →

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org!"
```

```
<bean id="myVehicle" class="vehicle.Bus">
```

```
<bean id="myVehicle2" class="vehicle.Car"> Name)
</bean>
```

```
</beans>
```

Now we can also
create object of
Car in Java code
by using this Id

③ Now create a new Main class for
Spring Configuration

main {

```
ApplicationContext context = new ClassPathXmlApp-
licationContext("config.xml");
Vehiclebean = context.getBean("myVehicle", Vehicle.class);

bean.getMilege
SysO(bean.getMilege);
```


* Dependency Injection →

Dependency Injection is a design pattern where the dependencies of a class are injected from outside rather than the class creating them itself.

Example →

instead of:

```
Service service = new Service();  
// Tightly coupled
```

we do:

```
Service service; // Spring will inject  
object  
// Loosely coupled.
```

Note → (why to use Dependency Injection)

i) Remove tight coupling between classes.

ii) Increase reusability.

iii) Enhance code maintainability.

Types of Dependency Injection →

two main types →

i) Constructor Injection

ii) setter Injection

iii) field Injection (learn afterwards while studying Annotation)

can be done by both
XML configuration
& Annotation config

① Constructor Injection ⇒

Inject dependencies using the class constructor.

Example →

① Address.java (class)

```
package com.example;
```

```
public class Address {
```

```
    private String city;
```

```
    public Address(String city) {
```

```
        this.city = city;
```

```
    }  
    public String getCity() {  
        return city;
```


ii student.java (class)

package com.example;

public class student {

private Address address;

public student(Address address) {

// constructor Injection.

this.address = address;

}

public void display() {

System.out.println("Student's Address: " +
address.getAddress());

}

}

③ config.xml (XML Configuration)

<?xml version="1.0" encoding="UTF-8"?>

<beans

—— // —— // ——

(copy from spring documentation
no need to remember)

>

<!-- Address Beans -->

<bean id="address" class="com.example.
Address">

<constructor-arg value="Mumbai"/>

</bean>

<!-- Student Bean with constructor injection -->

<bean id="student" class="com.example.student">

<constructor-arg ref="address"/>

</bean>

</beans>

Date: _____

OM OT OW OT OF OS OS

④ MainApp.java

```
public class MainApp {
```

```
    public static void Main (String[] args) {
```

```
        ApplicationContext context = new ClassPath-  
        XMLApplicationContext("config.xml");
```

```
        Student student = (Student) context.getBean("student");
```

```
        student.display();
```

```
    }  
}
```

② Setter Injection →

Inject dependencies using public setter methods.

Example →

① Address.java

```
package com.example;
```

```
public class Address {
```

```
    private String city;
```

```
    public void setCity(String city) {
```

```
        this.city = city;
```

```
    }
```

```
    public String getCity() {
```

```
        return city;
```

```
    }
```

```
}
```


Date:

OM OT OW OT OF OS OS

② Student.java

```
package com.example;
```

```
public class Student {  
    private Address address;
```

```
    // setter method for injection.  
    public void setAddress(Address address) {  
        this.address = address;  
    }
```

```
    public void display() {
```

```
        System.out.println("Student's Address: " +  
            address.getCity());
```

```
    }  
}
```

③ config.xml

```
<beans
```

```
>
```

⚡ - Address Bean -->

```
<bean id="address" class="com.example.  
    Address"/>
```



```
<property name = "city" value = "pune" />
</bean>
```

<!-- student Bean with setter Injection -->

```
<bean id = "student" class = "com.example.student">
```

```
<property name = "address" ref = "address" />
```

```
</bean>
```

```
</beans>
```

④ MainApp.java

```
public class MainApp {
```

```
    public static void main(String[] args) {
```

```
        ApplicationContext context = new
```

```
            ClassPathXmlApplicationContext("config.xml");
```

```
        Student student = (Student) context.getBean(
            "student");
```

```
        student.display();
```

```
    }
```


What is Spring Bean Scope →

In Spring, Bean Scope defines how many instances of a bean Spring container creates and how it manages them.

① Singleton →

Default scope, single shared instance

Only one instance per Spring container.
Shared across the entire app.

```
<bean id="myBean" class="com.example.MyBean"  
      scope="singleton"/>  
// by default no need to write
```

(In XML)

```
@Component("myBean")  
@Scope("singleton") // optional, it is default  
public class MyBean {  
}
```

(In Annotation)

ON OT OW OT OF OS OS

② Prototype →

A new instance is created every time the bean is requested.

<bean id="myBean" class="com.example.MyBean"
scope="prototype"/>

(In XML)

```
@Component("myBean")
@Scope("prototype")
public class MyBean {
}
```

(In Annotation)

③ Request Scope (web Apps only)

One bean instance per http request.

```
@Component("requestBean")
@Scope(value = WebApplicationContext.SCOPE_REQUEST)
```

```
public class MyBean {
```

```
}
```

(In Annotation)

<bean id="requestBean" class="com.ex.RegBean"
scope="request"/>

(In XML)

④ Session Scope → (only web Application)

one bean instance per http session.

<bean id="sessionBean" class="com.example.sessionBean" scope="session"/>
(XML based)

```
@Component("sessionBean")
@Scope(value = WebApplicationContext.SCOPE_SESSION)
public class MyBean {
```

```
}
```

(Annotation Based)

⑤ Application Scope → (only web App)

One bean instance per Servlet context-
(i.e per web app)

<bean id="appScopeBean" class="com.example.AppScopeBean" scope="application"/>
(XML)

```
@Component("appScopeBean")
@Scope(value = WebApplicationContext.SCOPE_APPLICATION)
public class MyBean {
```

(Annotation)

Example →

Testing Singleton vs Prototype

① Singleton :- →

```

MyBean bean1 = context.getBean("myBean");
MyBean bean2 = context.getBean("myBean");
    
```

```

System.out.println(bean1 == bean2); // true
    
```

② Prototype :- →

```

System.out.println(bean1 == bean2); // false
    
```

// if scope = "prototype"

* Annotation Based Configuration →

Lets understand with code

To create bean in Annotation based configuration we use "component" annotation

Code → (In package com.example.Vehicles;)

```
public interface Vehicle {
```

```
    public String getMileage();
```

```
}

//
// @Component
```

```
public class Car implements Vehicle {
```

```
    @Override
```

```
    public String getMileage() {
```

```
        return "50 KMPH";
```

```
    }
```

Here we can give name of bean like Component ("MyCar") but no need to give. it will be defaultly take it as a car (Car → car)
 suppose class name is CarService then it will be carService (CarService → carService)

Here still we need to create an config.xml file to contact with bean

In that config.xml we need to write
`<context:component-scan base-package =
 "com.engineeringDigest.vehicles"></context:
 component-scan>`
 package name where all beans available.

Spring will automatically create bean with the help of annotation.

iii) config.xml

`<beans`

copy from "spring documenta"

~~`</beans>`~~

`<context:component-scan base-package =
 "com.example.vehicles"></context:component-scan>`

`</beans>`

iv) main class

public class MySpringApp {

main method {

ApplicationContext context = new ClassPathApplic^{context}
 ("config.xml");

Vehicle bean = context.getBean("car", vehicle.class);
 syso (bean.getMileage);

Date: _____

OM OT OW OT OF OS OS

But if we did not want to create XML file then we can configure wing class as well

In this case we need to create one class in that class we need to give @Configuration Annotation.

Appconfig.java.

@Configuration

@ComponentScan("com.example")

// Enables annotation scanning

public class AppConfig {

}

{ package name
where beans
available }

* Dependency Injection using Annotation Based Configuration →

- i. Constructor Injection.
- ii. Setter Injection.
- iii. Field Injection.

① Address.java (common dependency class)

```
package com.example;
```

```
import org.springframework.stereotype.Component;
```

@Component

```
public class Address {
```

```
    public String getAddress () {
```

```
        return "Pune, Maharashtra";
```

```
    }
```

```
}
```